今天 上午4:14

我要用 Kaggle 的 Playground Series S4E5 Flood Prediction 資料集做多元線性回歸作業，
需遵循 CRISP-DM 流程。目標變數是 FloodProbability，有 20 個數值特徵。
請幫我寫 Google Colab 的 Python 程式：
1. 用一段文字說明 Business Understanding（預測洪水機率的商業/社會價值）
2. 讀取 train.csv，顯示資料筆數、欄位、資料型態、缺失值、描述統計
3. 畫出目標變數分布圖、特徵相關係數熱力圖、各特徵與目標的相關係數長條圖
請加上中文註解，圖表標題用英文避免字型問題。

處理時間為 8s

以下程式適用於 **Google Colab**，使用 Kaggle Playground Series S4E5：*Regression with a Flood Prediction Dataset*，按照 CRISP-DM 方法進行分析。

已核對 Kaggle 官方競賽資料：目標是預測 `FloodProbability`，評估指標為 R^2，資料屬於合成資料（Synthetic Data）。 k Kaggle +1

本次先完成 CRISP-DM 的前三個階段：

1. Business Understanding（商業理解）
2. Data Understanding（資料理解）
3. Data Preparation（初步特徵選取及資料檢查）

並繪製三張探索性資料分析圖表（EDA），供後續多元線性回歸使用。

1. Business Understanding

以下文字可直接放入 Colab 的 Markdown 儲存格，作為作業報告。

Business Understanding — 洪水機率預測

洪水是常見的自然災害之一，可能造成生命財產損失、基礎設施損壞及經濟活動中斷。本研究使用 Kaggle Playground Series S4E5 洪水預測資料集，以 FloodProbability 作為目標變數，透過 20 個環境、氣候及人為因素建立多元線性回歸模型，分析各項因素與洪水機率之間的關係，並預測不同條件下的洪水風險。

本研究的商業與社會價值在於協助政府及相關單位理解潛在的洪水風險因素，作為防災規劃、基礎設施改善及資源配置的分析參考。研究將透過資料探索、特徵分析及模型評估，了解多元線性回歸在洪水機率預測問題上的適用性。

研究限制： 本資料集為合成資料，模型結果主要用於機器學習方法的研究與教學，不宜直接作為真實洪水預警或公共政策決策依據。

2. Data Understanding

Cell 1 — 匯入套件

Python 執行

```python
# ========================================
# CRISP-DM: Data Understanding
# 1. Import Libraries
# ========================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from IPython.display import display

# 設定圖表風格
sns.set_theme(style="whitegrid")

# 設定圖表大小與解析度
plt.rcParams["figure.figsize"] = (10, 6)
plt.rcParams["figure.dpi"] = 120

print("Libraries imported successfully!")
```

Cell 2 — 上傳並讀取 train.csv

先從 Kaggle S4E5 資料下載頁面 ↗ 取得 `train.csv`，執行以下程式後上傳即可。

Python 執行

```python
# ========================================
# 2. Load Dataset
# ========================================

from google.colab import files
import io

# 上傳 Kaggle train.csv
uploaded = files.upload()
```

```python
# 確認檔案名稱
if "train.csv" not in uploaded:
    raise FileNotFoundError("Please upload train.csv")

# 讀取 CSV
df = pd.read_csv(io.BytesIO(uploaded["train.csv"]))

# 顯示資料維度
print("Dataset Shape:", df.shape)
print("Number of Rows:", df.shape[0])
print("Number of Columns:", df.shape[1])

# 顯示前五筆資料
print("\nFirst 5 Rows:")
display(df.head())
```

Cell 3 — 欄位、資料型態與缺失值

Python 執行

```python
# ========================================
# 3. Dataset Information
# ========================================

# 顯示所有欄位名稱
print("Column Names:")
print(df.columns.tolist())

# 顯示資料基本資訊
print("\nDataset Information:")
df.info()

# 整理各欄位的資料型態與缺失值
data_info = pd.DataFrame({
    "Data Type": df.dtypes.astype(str),
    "Missing Values": df.isnull().sum(),
    "Missing Percentage (%)":
        df.isnull().mean() * 100,
    "Unique Values": df.nunique()
})

# 顯示資料品質摘要
print("\nData Quality Summary:")
display(data_info)

# 檢查重複資料（不包含 id）
print("\nDuplicated Rows (excluding ID):")
print(df.drop(columns=["id"], errors="ignore")
        .duplicated().sum())
```

Cell 4 — 描述統計（Descriptive Statistics）

Python 執行

```python
# ======================================
# 4. Descriptive Statistics
# ======================================

# 顯示數值欄位的描述統計
print("Descriptive Statistics:")
display(df.describe().T)

# 顯示目標變數的統計資訊
target = "FloodProbability"

print("\nTarget Variable Statistics:")
print("Mean:", df[target].mean())
print("Median:", df[target].median())
print("Standard Deviation:", df[target].std())
print("Minimum:", df[target].min())
print("Maximum:", df[target].max())
```

3. Data Preparation — 初步特徵選取

原始資料中的 `id` 是資料識別碼，不是用來描述洪水風險的解釋變數，因此不應納入多元線性回歸。

Cell 5 — 定義 X 與 y

Python 執行

```python


).select_dtypes(include=[np.number]).columns.tolist()

# 自變數 X：20 個數值特徵
X = df[feature_cols]

# 應變數 y：洪水機率
```

```python
y = df[target]

# 檢查特徵數量是否正確
assert X.shape[1] == 20, "Expected 20 features"
assert X.columns.is_unique, "Duplicate feature names found"
assert y.notna().all(), "Target contains missing values"

print("Number of Features:", X.shape[1])
print("Target Variable:", target)

print("\nFeature Names:")
for i, feature in enumerate(feature_cols, 1):
    print(f"{i:2d}. {feature}")

print("\nX Shape:", X.shape)
print("y Shape:", y.shape)

# 檢查特徵是否含有缺失值
print("\nTotal Missing Values in X:")
print(X.isnull().sum().sum())
```

此處僅完成初步特徵選取，尚未進行訓練集與驗證集切分，也未進行缺失值填補或標準化。

4. Exploratory Data Analysis（EDA）

Cell 6 — FloodProbability 分布圖

使用 Histogram 搭配 KDE 曲線，觀察目標變數的集中程度與分布形狀。

Python 執行

```python
# ========================================
# 6. Target Variable Distribution
# ========================================

plt.figure(figsize=(10, 6))

sns.histplot(
    data=df,
    x="FloodProbability",
    bins=50,
    kde=True,
    color="steelblue"
)

plt.title("Distribution of Flood Probability")
plt.xlabel("Flood Probability")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

分析重點

觀察洪水機率是否集中在特定區間、是否有明顯偏態，以及是否存在極端數值。這些結果有助於了解預測目標的資料特性，但不能單憑目標變數分布判斷線性回歸是否適用。

Cell 7 — 特徵相關係數熱力圖

使用 Pearson Correlation，分析 20 個特徵彼此之間的線性關係，作為初步檢查多重共線性（Multicollinearity）的依據。

Python 執行

```python
# ========================================
# 7. Feature Correlation Heatmap
# ========================================

# 計算 20 個特徵的 Pearson 相關係數
corr_matrix = X.corr(method="pearson")

# 繪製相關係數熱力圖
plt.figure(figsize=(16, 12))

sns.heatmap(
    corr_matrix,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    center=0,
    vmin=-1,
    vmax=1,
    linewidths=0.5,
    square=True
)

plt.title("Feature Correlation Heatmap")
plt.xticks(rotation=90)
plt.yticks(rotation=0)

plt.tight_layout()
plt.show()
```

分析重點

Pearson 相關係數範圍為 -1 至 1：

- 接近 1：高度正向線性相關。
- 接近 -1：高度負向線性相關。
- 接近 0：沒有明顯的線性相關。

若多個特徵彼此高度相關，可能造成多元線性回歸係數不穩定。熱力圖只能作初步檢查，後續可使用 VIF 進一步評估。

Cell 8 — 各特徵與 FloodProbability 的相關係數長條圖

這張圖可以幫助辨識哪些變數與目標變數具有較強的單變量線性關係。

Python 執行

```python
# ======================================
# 8. Feature-Target Correlation
# ======================================

# 計算每個特徵與目標變數的 Pearson 相關係數
target_corr = (
    df[feature_cols + [target]]
    .corr(method="pearson")[target]
    .drop(target)
    .sort_values(ascending=False)
)

# 依正負相關設定顏色
colors = [
    "steelblue" if value >= 0 else "salmon"
    for value in target_corr.values
]

# 繪製水平長條圖
plt.figure(figsize=(11, 9))

plt.barh(
    target_corr.index,
    target_corr.values,
    color=colors
)

plt.axvline(x=0, color="black", linewidth=0.8)

plt.title("Feature Correlation with Flood Probability")
plt.xlabel("Pearson Correlation Coefficient")
plt.ylabel("Features")

# 顯示數值標籤
for i, value in enumerate(target_corr.values):
    offset = 0.002 if value >= 0 else -0.002
    plt.text(
        value + offset,
        i,
        f"{value:.3f}",
        va="center",
        ha="left" if value >= 0 else "right",
        fontsize=9
    )
```

```
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

# 顯示完整相關係數表
print("Feature-Target Correlation:")
display(target_corr.to_frame(name="Correlation"))
```

分析重點

這張圖反映的是各特徵與 `FloodProbability` 的**單變量 Pearson 相關性**，不是多元線性回歸模型中的迴歸係數，也不能直接解釋為因果影響或模型特徵重要性。

---

5. CRISP-DM 作業進度

階段	本次完成內容	狀態
1. Business Understanding	研究目的、社會價值、資料限制	✅
2. Data Understanding	資料結構、型態、缺失值、統計、EDA	✅
3. Data Preparation	排除 ID、定義 20 個 X 與 y	✅ 初步完成
4. Modeling	Multiple Linear Regression	尚未執行
5. Evaluation	MAE、MSE、RMSE、R^2、殘差分析	尚未執行
6. Deployment	模型應用與結果展示	尚未執行

後續建議： 這份資料集很適合接著實作 `sklearn.linear_model.LinearRegression`，將資料切分為 80% 訓練集與 20% 測試集，使用 MAE、RMSE 和 R^2 評估預測表現。由於這是合成資料，分析時應避免將特徵名稱直接視為已驗證的真實洪水因果因素。

上述程式已針對 Kaggle S4E5 的
宣稱相關係數數值、圖表形狀或

> 接續上面，請幫我做 Data Preparation：
> 1. 刪除 id 欄位，檢查並處理缺失值與重複值
> 2. 用 train_test_split 切成 80% 訓練、20% 測試（random_state=42）
> 3. 用 StandardScaler 標準化（只用訓練集 fit）
> 4. 簡單說明每一步為什麼要這樣做

以下接續前面的 Google Colab 程式，完成 **CRISP-DM 第三階段：Data Preparation（資料前處理）**，並確保後續可以直接進行 Multiple Linear Regression。

處理流程為：

Raw Dataset（`df`）

↓

Remove ID & Duplicates

↓

Train / Test Split (80% / 20%)

↓

Missing Value Imputation (Training Median)

↓

StandardScaler (Training Fit Only)

↓

Ready for Modeling

Cell 9 — 刪除 ID、檢查缺失值與重複值

Python 執行

```python
# ======================================
# CRISP-DM: Data Preparation
# 9. Data Cleaning
# ======================================

# 建立副本，保留原始資料
df_clean = df.copy()

# 刪除 id 欄位
df_clean = df_clean.drop(
    columns=["id"],
    errors="ignore"
)

# 檢查缺失值
print("Missing Values Before Processing:")
print(df_clean.isnull().sum())

# 檢查重複資料
duplicates = df_clean.duplicated().sum()
print("\nDuplicated Rows:", duplicates)

# 刪除完全重複的資料列
df_clean = df_clean.drop_duplicates().reset_index(drop=True)

print("\nDataset Shape After Cleaning:", df_clean.shape)

# 定義目標與特徵
target = "FloodProbability"
```

```python
# 若目標變數缺失，該筆資料無法用於監督式學習
df_clean = df_clean.dropna(subset=[target])

X = df_clean.drop(columns=[target])
y = df_clean[target]

print("Number of Features:", X.shape[1])
print("Remaining Missing Values in Features:",
      X.isnull().sum().sum())
```

為什麼？

`id` 是資料識別碼，不代表洪水風險因素，因此不納入模型。完全重複的資料列可能讓相同樣本在模型中被重複計算，因此先刪除；目標值缺失的資料無法用來訓練監督式回歸模型。

不過，這裡不立即填補特徵的缺失值，因為填補所需的統計量必須從訓練集計算，避免 Data Leakage（資料洩漏）。

Cell 10 — 切分訓練集與測試集

Python 執行

```python
# ========================================
# 10. Train-Test Split
# ========================================

from sklearn.model_selection import train_test_split

# 80% 訓練資料、20% 測試資料
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training Set:", X_train.shape)
print("Testing Set:", X_test.shape)

print("Training Target:", y_train.shape)
print("Testing Target:", y_test.shape)
```

為什麼？

- **80% 訓練集：** 用來訓練多元線性回歸模型。
- **20% 測試集：** 用於評估模型對未見資料的預測能力。
- **random_state=42：** 固定隨機切分結果，讓每次執行的資料分組一致，確保實驗可重現。

Cell 11 — 處理特徵缺失值

使用 `SimpleImputer`，以訓練集各欄位的中位數（Median）填補缺失值。

Python 執行

```python
# ======================================
# 11. Missing Value Imputation
# ======================================

from sklearn.impute import SimpleImputer

# 使用中位數填補數值特徵缺失值
imputer = SimpleImputer(strategy="median")

# 只能使用訓練集計算中位數
X_train_imputed = imputer.fit_transform(X_train)

# 測試集使用相同的填補規則
X_test_imputed = imputer.transform(X_test)

# 轉回 DataFrame，保留欄位名稱
X_train_imputed = pd.DataFrame(
    X_train_imputed,
    columns=X_train.columns,
    index=X_train.index
)

X_test_imputed = pd.DataFrame(
    X_test_imputed,
    columns=X_test.columns,
    index=X_test.index
)

# 確認缺失值已處理
print("Training Missing Values:",
      X_train_imputed.isnull().sum().sum())

print("Testing Missing Values:",
      X_test_imputed.isnull().sum().sum())
```

為什麼？

多元線性回歸模型一般無法直接處理缺失值。中位數相較於平均數不容易受到極端值影響。

`fit_transform()` 只對訓練集執行，測試集僅使用 `transform()`，確保模型前處理不會提前取得測試資料的統計資訊。

Cell 12 — StandardScaler 標準化

Python 執行

```python
# ======================================
# 12. Feature Standardization
# ======================================

from sklearn.preprocessing import StandardScaler

# 建立標準化工具
scaler = StandardScaler()

# 使用訓練集計算平均數與標準差
X_train_scaled = scaler.fit_transform(X_train_imputed)

# 測試集套用相同的標準化參數
X_test_scaled = scaler.transform(X_test_imputed)

# 轉為 DataFrame
X_train_scaled = pd.DataFrame(
    X_train_scaled,
    columns=X_train.columns,
    index=X_train.index
)

X_test_scaled = pd.DataFrame(
    X_test_scaled,
    columns=X_test.columns,
    index=X_test.index
)

# 顯示標準化後的資料
print("Standardized Training Data:")
display(X_train_scaled.head())

# 檢查訓練集各欄位的平均值與標準差
print("\nMean After Scaling:")
print(X_train_scaled.mean().round(4))

print("\nStandard Deviation After Scaling:")
print(X_train_scaled.std(ddof=0).round(4))
```

為什麼？

StandardScaler 會將每個特徵轉換成平均值約為 0、標準差約為 1 的尺度。

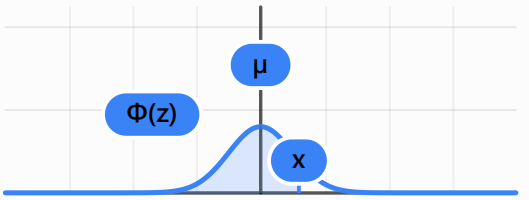
$$z = \frac{x-\mu}{\sigma}$$

$$z = \frac{(1.2)-(0)}{1} = 1.2$$

$$\Phi(z) \approx 88.5\%$$

x 1.2

μ 0.0



提供回饋

這有助於比較標準化後的迴歸係數，也方便日後使用 Ridge、Lasso 等模型。

需要注意的是，**一般未正則化的多元線性回歸並不強制需要標準化**。在含截距、相同特徵資訊且數值計算穩定的情況下，標準化通常不會改變其理論預測結果，但會改變迴歸係數的尺度。

此外，測試集必須使用訓練集計算出的平均值與標準差，不能重新執行 `fit()`，否則會造成資料洩漏。

Cell 13 — 最終資料檢查

Python 執行

```python
# ========================================
# 13. Final Data Verification
# ========================================

print("=== Data Preparation Summary ===")

print("Training Features:", X_train_scaled.shape)
print("Testing Features:", X_test_scaled.shape)

print("Training Target:", y_train.shape)
print("Testing Target:", y_test.shape)

# 驗證特徵數量
assert X_train_scaled.shape[1] == 20

# 驗證缺失值
assert not X_train_scaled.isnull().any().any()
assert not X_test_scaled.isnull().any().any()

# 驗證資料沒有無限值
assert np.isfinite(X_train_scaled.to_numpy()).all()
assert np.isfinite(X_test_scaled.to_numpy()).all()

print("\nData Preparation Completed Successfully!")
```

CRISP-DM — Data Preparation 報告摘要

本研究首先移除不具預測意義的 id 欄位，並檢查資料中的缺失值與重複紀錄，刪除完全重複的資料及目標變數缺失的樣本。接著使用 train_test_split，依照 80% 與 20% 的比例將資料分為訓練集與測試集，並設定 random_state=42 以確保結果可重現。

針對特徵缺失值，使用 SimpleImputer 依據訓練集的中位數進行填補。最後透過 StandardScaler 進行特徵標準化，使訓練集各特徵的平均值約為 0、標準差約為 1。所有需學習資料統計參數的前處理程序均僅使用訓練集進行 fit，再套用至測試集，避免資料洩漏，完成後即可進入多元線性回歸模型建立階段。

下一階段 Modeling 可以直接使用 `X_train_scaled`、`X_test_scaled`、`y_train`、`y_test` 建立 `LinearRegression()`，再利用 MAE、MSE、RMSE、R^2 及殘差圖進行模型評估。

方法學提醒： 以上採用先全資料去除完全重複列、再切分的流程，適合這份課堂作業。若目的是嚴格比較模型泛化能力，仍應依資料的產生機制檢查重複樣本，並考慮是否存在相同來源樣本跨越訓練集與測試集的問題。

請幫我做特徵選擇，比較三種方法，只用訓練資料：
1. SelectKBest (f_regression)
2. RFE (搭配 LinearRegression)
3. LassoCV（看哪些係數變成 0）
分別選出 k = 5, 10, 15, 20 個特徵，用 5-fold 交叉驗證比較 R^2，畫出「特徵數 vs R^2」折線圖，並用表格列出每個方法選到的特徵。最後建議選哪一組特徵並說明理由。

處理時間為 1m 7s

CRISP-DM — Feature Selection（特徵選擇）

接續前面的 Data Preparation，本階段比較三種特徵選擇方法，並以 **5-Fold Cross-Validation 的平均 R^2** 判斷哪種方法、哪個特徵數量適合多元線性回歸。

1. 三種方法比較

方法	特徵選擇原理	特點
SelectKBest（`f_regression`）	根據各特徵與目標之間的單變量線性關係進行排序	速度快，但不考慮其他特徵
RFE（`LinearRegression`）	反覆訓練模型並移除較不重要的特徵	考慮多變量模型中的係數
LassoCV	使用 L1 正則化，使部分係數縮小至 0	可進行自動特徵篩選

本次比較的特徵數量為 `k = [5, 10, 15, 20]`，共 12 組實驗。

兩項重要設計：

- 只使用先前建立的 `X_train`、`y_train` 進行特徵選擇及交叉驗證。`X_test`、`y_test` 保留到最後模型評估。

- LassoCV 會自動決定非零係數的數量，因此為了公平比較固定的 k，本次以 LassoCV 係數絕對值進行排序，再選取前 k 個特徵；另外呈現真正的非零係數數量。

以下 Colab 程式從 **Cell 14** 接續執行。

2. Colab 程式碼

Cell 14 — 匯入套件與設定參數

Python 執行

```python
# ======================================
# CRISP-DM: Feature Selection
# 14. Import Libraries and Settings
# ======================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import warnings

from sklearn.model_selection import KFold
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest, f_regression, RFE
from sklearn.linear_model import LinearRegression, LassoCV
from sklearn.metrics import r2_score
from sklearn.exceptions import ConvergenceWarning
from IPython.display import display

# 實驗設定
k_values = [5, 10, 15, 20]
methods = ["SelectKBest", "RFE", "LassoCV"]

cv = KFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

# 使用未標準化的訓練集
# 標準化會在每個 CV fold 內重新執行
X_fs = X_train.copy()
y_fs = y_train.copy()

assert X_fs.shape[1] == 20
assert X_fs.columns.is_unique
assert y_fs.notna().all()

feature_names = X_fs.columns.to_numpy()

# LassoCV 的 Alpha 搜尋範圍
lasso_alphas = np.logspace(-5, -0.5, 40)
```

```
print("Feature Selection Settings")
print("Methods:", methods)
print("Feature Counts:", k_values)
print("CV Folds:", cv.get_n_splits())
print("Training Samples:", len(X_fs))
```

這裡刻意使用 `X_train` 而非前一步已完成標準化的 `X_train_scaled`。原因是交叉驗證每一折都應重新計算中位數、平均值及標準差，避免驗證折的統計資訊進入特徵選擇與模型訓練。

Cell 15 — 建立特徵選擇函式

Python 執行

```python
# ======================================
# 15. Feature Selection Functions
# ======================================

def select_features(method, k, X_data, y_data, lasso_model=None):
    """
    根據指定方法選擇 k 個特徵
    回傳：選取欄位的 Boolean mask
    """

    if method == "SelectKBest":

        selector = SelectKBest(
            score_func=f_regression,
            k=k
        )
        selector.fit(X_data, y_data)
        return selector.get_support()

    elif method == "RFE":

        selector = RFE(
            estimator=LinearRegression(),
            n_features_to_select=k,
            step=1
        )
        selector.fit(X_data, y_data)
        return selector.get_support()

    elif method == "LassoCV":

        # LassoCV 先選最佳 Alpha
        # 再依係數絕對值排序取前 k 名
        if lasso_model is None:
            raise ValueError("Lasso model is required")

        importance = np.abs(lasso_model.coef_)
```

```
        # 相同係數以原始欄位順序決定排序
        ranking = np.argsort(-importance, kind="stable")

        selected = ranking[:k]

        mask = np.zeros(
            X_data.shape[1],
            dtype=bool
        )
        mask[selected] = True

        return mask

    else:
        raise ValueError("Unknown method")
```

LassoCV 特別注意： 此函式為了固定 k 的公平比較，使用 LassoCV 係數絕對值排名。因此即使某些係數為 0，也可能在 k 較大時納入排名。這不等同於 Lasso 自動選出的非零特徵集合，兩種結果會分別呈現。

Cell 16 — 執行 5-Fold Cross-Validation

這是主要計算部分。每一折都只用該折的訓練資料進行特徵選擇，再以驗證資料計算 R^2。

Python 執行

```
# ========================================
# 16. Five-Fold Cross-Validation
# ========================================

results = []

# 儲存每個方法及 k 的五折 R²
cv_scores = {
    (method, k): []
    for method in methods
    for k in k_values
}

# 外層 5-Fold Cross-Validation
for fold, (train_idx, val_idx) in enumerate(
    cv.split(X_fs), start=1
):

    print(f"Processing Fold {fold}/5")

    X_tr = X_fs.iloc[train_idx]
    X_val = X_fs.iloc[val_idx]

    y_tr = y_fs.iloc[train_idx]
    y_val = y_fs.iloc[val_idx]
```

```
    # 只使用該折訓練資料計算填補值
    imputer_fold = SimpleImputer(strategy="median")

    X_tr_imp = imputer_fold.fit_transform(X_tr)
    X_val_imp = imputer_fold.transform(X_val)

    # 只使用該折訓練資料計算標準化參數
    scaler_fold = StandardScaler()
```

執行時間提醒：S4E5 訓練資料量大，RFE 與 LassoCV 可能需要較長時間。此版本優先保留完整訓練資料，不以抽樣取代正式交叉驗證。

3. 結果視覺化

Cell 17 — 特徵數 vs R^2 折線圖

Python 執行

```python
# ======================================
# 17. Feature Count vs R2
# ======================================

plt.figure(figsize=(10, 6))

for method in methods:

    subset = results_df[
        results_df["Method"] == method
    ].sort_values("K")

    plt.errorbar(
        subset["K"],
        subset["Mean R2"],
        yerr=subset["SE R2"],
        marker="o",
        linewidth=2,
        capsize=4,
        label=method
    )

plt.title("Number of Features vs Cross-Validation R2")
plt.xlabel("Number of Selected Features")
plt.ylabel("Mean CV R2 Score")
plt.xticks(k_values)

plt.legend(title="Selection Method")
plt.grid(alpha=0.3)
plt.tight_layout()

plt.show()
```

圖中每條線代表一種特徵選擇方法，點為平均 R^2，誤差棒為五折 R^2 的標準誤（SE）。

Cell 18 — 整理 R^2 比較表

Python 執行

```python
# ======================================
# 18. R2 Comparison Table
# ======================================
```

```python
# 將比較結果整理成矩陣
comparison_table = results_df.pivot(
    index="K",
    columns="Method",
    values="Mean R2"
)

print("Mean Cross-Validation R2:")
display(comparison_table.round(6))

# 顯示完整排名
ranking_df = results_df.sort_values(
    by="Mean R2",
    ascending=False
).reset_index(drop=True)

print("\nModel Ranking:")
display(ranking_df.round(6))
```

4. 列出每種方法選取的特徵

Cell 19 — 使用完整訓練集重新進行特徵選擇

前面的 5-Fold CV 是用來評估不同方法與 k 的效果。現在使用完整的 80% 訓練資料選出最終特徵名稱，仍不使用測試集。

Python 執行

```python
# ======================================
# 19. Final Feature Selection
# ======================================

# 只使用完整訓練集 fit 前處理
final_imputer = SimpleImputer(strategy="median")
X_train_imp = final_imputer.fit_transform(X_fs)

final_scaler = StandardScaler()
X_train_s = final_scaler.fit_transform(X_train_imp)

# 用完整訓練集建立 LassoCV
final_lasso = LassoCV(
    alphas=lasso_alphas,
    cv=5,
    max_iter=4000,
    n_jobs=-1,
    random_state=42
)

final_lasso.fit(X_train_s, y_fs)

# 儲存每組方法選到的特徵
```

```python
feature_results = []
selected_features_dict = {}

for method in methods:
    for k in k_values:

        mask = select_features(
            method,
            k,
            X_train_s,
            y_fs,
            lasso_model=final_lasso
```

此表會顯示三種方法各自選取 5、10、15、20 個特徵的名稱，共 12 組結果。

Cell 20 — 查看 LassoCV 非零係數

Python 執行

```python
# ========================================
# 20. LassoCV Coefficient Analysis
# ========================================

# 建立係數表
lasso_coef_df = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": final_lasso.coef_
})

# 標記非零係數
# atol 用來處理浮點運算中的極小數值
lasso_coef_df["Nonzero"] = ~np.isclose(
    lasso_coef_df["Coefficient"],
    0,
    atol=1e-8,
    rtol=0
)

# 依絕對係數大小排序
lasso_coef_df["Abs Coefficient"] = (
    lasso_coef_df["Coefficient"].abs()
)
```

```python
lasso_coef_df = lasso_coef_df.sort_values(
    "Abs Coefficient",
    ascending=False
)

print("Best Lasso Alpha:", final_lasso.alpha_)

print("Nonzero Coefficients:",
      lasso_coef_df["Nonzero"].sum())

print("Zero Coefficients:",
```

這張圖可以觀察 LassoCV 是否真的將某些係數壓縮至 0。若最佳 alpha 很小，LassoCV 也可能保留全部 20 個特徵，這並不是程式錯誤。

5. 自動推薦最佳特徵組合

我建議不要只選擇平均 R^2 最高的一組，而是加入模型簡潔性的考量。

這裡採用 **One-Standard-Error Rule（1-SE Rule）** 作為實務上的選擇準則：在最佳平均 R^2 減去其標準誤所形成的容許範圍內，優先選擇特徵數較少的模型。

Cell 21 — 自動選擇與輸出建議

Python 執行

```python
# ========================================
# 21. Model Recommendation
# ========================================

# 取得平均 R2 最高的組合
```

```
best_row = results_df.loc[
    results_df["Mean R2"].idxmax()
]

# 最佳模型的 1-SE 門檻
threshold = best_row["Mean R2"] - best_row["SE R2"]

# 篩選落在 1-SE 範圍內的候選組合
candidates = results_df[
    results_df["Mean R2"] >= threshold
].copy()

# 優先選較少特徵，同數量時選較高 R2
recommended = candidates.sort_values(
    by=["K", "Mean R2"],
    ascending=[True, False]
).iloc[0]

best_method = recommended["Method"]
best_k = int(recommended["K"])

best_features = selected_features_dict[
    (best_method, best_k)
]

print("=== Highest Mean R2 ===")
print("Method:", best_row["Method"])
print("K:", int(best_row["K"]))
print("Mean R2:", round(best_row["Mean R2"], 6))

print("\n=== Recommended Feature Set ===")
print("Method:", best_method)
print("K:", best_k)
print("Mean R2:", round(recommended["Mean R2"], 6))

print("\nSelected Features:")
for i, feature in enumerate(best_features, 1):
    print(f"{i}. {feature}")

print("\nRecommendation Reason:")
print(
    "Selected the smallest feature set within "
    "one standard error of the best mean CV R2."
)
```

注意：1-SE Rule 是一種兼顧模型效能與複雜度的啟發式決策準則，不代表候選模型在統計上已被證明等效。

6. 作業報告結論

以下內容可放入 Colab Markdown，執行程式後再填入實際數據。

Feature Selection — Results and Discussion

本研究採用 SelectKBest（f_regression）、RFE（LinearRegression）及 LassoCV 三種方法進行特徵選擇，分別比較 5、10、15、20 個特徵，共計 12 組模型配置。

為降低資料洩漏風險，所有特徵選擇及資料標準化程序均於交叉驗證的訓練折內執行，並採用 5-Fold Cross-Validation，以平均 R^2 作為模型效能評估指標。

實驗結果顯示，平均 R^2 最高的組合為 **[填入方法]**，使用 **[填入特徵數量]** 個特徵，平均 R^2 為 **[填入數值]**。

綜合考量預測效能與模型複雜度，本研究進一步採用 One-Standard-Error Rule 選擇較精簡的特徵組合，建議使用 **[填入建議方法]** 所選出的 **[填入特徵數量]** 個特徵建立多元線性回歸模型。

此方法可在維持接近最佳交叉驗證表現的同時，降低模型複雜度，並使後續迴歸係數分析更容易解釋。

7. 結果解讀與注意事項

1. **目前無法直接指定哪組最佳。** 尚未取得你執行程式後的 R^2，因此不應預先斷言哪種方法勝出。
2. **20 個特徵的組合是必要的基準組。** 因為全部特徵都被保留，三種方法在 k=20 時，經由相同的 LinearRegression 評估，理論上應得到相同的 CV R^2（數值誤差除外）。
3. **LassoCV 的係數為零不代表該特徵完全沒有影響。** 係數受到正則化強度及其他特徵影響。
4. **本次最佳組合是依 5-fold CV 選出的。** 因為同一組交叉驗證結果也用來挑選方法與 k，獲選組合的 CV R^2 仍可能偏樂觀；正式效能應在未使用過的 20% 測試集上評估。

下一階段： 使用 `best_feature` 算 MAE、MSE、RMSE、R^2，

用選出的特徵建立模型並比較：
1. LinearRegression（sklearn）
2. Ridge、Lasso
3. 用 statsmodels OLS 再建一次，印出 summary（係數、p-value、R^2、Adjusted R^2）

並解釋前 5 個最重要特徵的係數意義。

CRISP-DM — Modeling（多元線性回歸模型建立）

接續前面的 Feature Selection，使用選出的 `best_features` 建立四個模型：

1. **LinearRegression**：一般最小平方法（Ordinary Least Squares）
2. **Ridge Regression**：L2 Regularization
3. **Lasso Regression**：L1 Regularization
4. **statsmodels OLS**：分析迴歸係數、p-value、R^2 與 Adjusted R^2

所有模型使用相同的特徵與訓練 / 測試資料，確保比較公平。

1. 模型比較原理

模型	方法	主要用途

LinearRegression	最小化殘差平方和	基準模型
Ridge	殘差平方和 + L2 懲罰	抑制過大的係數
Lasso	殘差平方和 + L1 懲罰	可將部分係數縮減至 0
statsmodels OLS	普通最小平方法	統計推論與係數解釋

注意： sklearn 的 `LinearRegression` 與 statsmodels 的 `OLS`，在使用相同資料、截距設定及完整秩設計矩陣的情況下，理論上會得到相同的迴歸預測。OLS 並不是不同的預測演算法，這裡額外建立一次是為了取得統計分析結果。

2. Colab 程式

Cell 22 — 準備選出的特徵

沿用先前產生的 `best_features`、`X_train`、`X_test`、`y_train`、`y_test`。

Python 執行

```python
# ========================================
# CRISP-DM: Modeling
# 22. Prepare Selected Features
# ========================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm

from IPython.display import display
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import (
    LinearRegression, RidgeCV, LassoCV
)
from sklearn.model_selection import KFold
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# 使用前一階段選出的最佳特徵
selected_features = list(best_features)

# 檢查特徵數量
assert len(selected_features) > 0
assert len(selected_features) == len(set(selected_features))
assert set(selected_features).issubset(X_train.columns)
assert set(selected_features).issubset(X_test.columns)
```

```python
# 選取相同特徵
X_train_selected = X_train[selected_features].copy()
```

說明： 只使用前面交叉驗證選定的特徵，不重新使用測試資料選特徵，避免測試資料洩漏。

Cell 23 — 建立 LinearRegression、Ridge、Lasso

這裡使用 `Pipeline`，確保缺失值填補及標準化都只從訓練集學習。Ridge 與 Lasso 的正則化參數 alpha，另外透過訓練集內部的 5-fold CV 選擇。

Python 執行

```
    # 評估指標
    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)

    # 取得最佳 Alpha
    alpha = getattr(
        pipeline.named_steps["model"],
        "alpha_",
        np.nan
    )

    results.append({
        "Model": name,
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R2": r2,
        "Alpha": alpha
    })

# 顯示模型比較表
results_df_model = pd.DataFrame(results)

print("Model Comparison (Test Set):")
display(results_df_model.round(6))
```

這張表會呈現：

- **MAE、MSE、RMSE：** 越低越好。
- **R^2：** 越接近 1，代表模型對測試集目標變數的解釋程度越高；R^2 也可能為負值。
- **Alpha：** Ridge 和 Lasso 的正則化強度；LinearRegression 不使用 alpha。

此處的 RidgeCV 和 LassoCV 使用各自的內部交叉驗證選擇 alpha。對 LassoCV 而言，交叉驗證主要依預測均方誤差進行選擇，而不是直接最佳化 R^2。

Cell 24 — 模型比較長條圖

Python 執行

```
# ======================================
# 24. Model Performance Comparison
# ======================================

plt.figure(figsize=(9, 5))

bars = plt.bar(
    results_df_model["Model"],
    results_df_model["R2"],
    color=["steelblue", "seagreen", "darkorange"]
)

plt.title("Regression Model Comparison")
plt.xlabel("Regression Model")
plt.ylabel("Test R2 Score")
plt.axhline(0, color="black", linewidth=0.8)

for bar in bars:
    value = bar.get_height()
    plt.annotate(
        f"{value:.5f}",
        (bar.get_x() + bar.get_width() / 2, value),
        xytext=(0, 5 if value >= 0 else -12),
        textcoords="offset points",
        ha="center",
        fontsize=10
    )

plt.tight_layout()
plt.show()
```

3. statsmodels OLS 統計分析

Cell 25 — 建立 OLS 並顯示 Summary

OLS 使用與 LinearRegression 相同的訓練集、特徵及標準化參數，使兩個模型的係數與預測可以直接比較。

Python 執行

```
# ======================================
# 25. Statsmodels OLS Regression
# ======================================
```

```
# 取得 LinearRegression 的前處理工具
linear_pipeline = trained_models["LinearRegression"]

imputer = linear_pipeline.named_steps["imputer"]
scaler = linear_pipeline.named_steps["scaler"]

# 使用已由訓練集 fit 的前處理工具
X_train_ols = scaler.transform(
    imputer.transform(X_train_selected)
)

X_test_ols = scaler.transform(
    imputer.transform(X_test_selected)
)

# 轉成 DataFrame，保留特徵名稱
X_train_ols = pd.DataFrame(
    X_train_ols,
    columns=selected_features,
    index=X_train_selected.index
)

X_test_ols = pd.DataFrame(
    X_test_ols,
    columns=selected_features,
    index=X_test_selected.index
)

# 加入截距項
X_train_ols = sm.add_constant(
    X_train_ols,
```

如何解讀 OLS Summary？

指標	意義

`coef`	其他變數固定下，該特徵增加一單位時的目標預測變化量
`P>|t|`	在虛無假設（係數為 0）及模型假設成立下，衡量觀察結果的統計顯著性
`R-squared`	訓練資料中，模型解釋的目標變異比例
`Adj. R-squared`	考慮特徵數量後調整的 R^2
`std err`	係數估計的標準誤
`[0.025, 0.975]`	係數的 95% 信賴區間

要特別區分：**OLS Summary 顯示的 R^2 是訓練集 R^2**，不是前面模型比較表中的測試集 R^2。

另外，因為這些特徵已經根據訓練資料進行選擇，OLS 原始 p-value 沒有校正特徵選擇造成的推論偏差，不能直接視為嚴格有效的顯著性證據。

4. 前五個重要特徵及係數解釋

本研究以 OLS 的**標準化自變數係數絕對值**排序，找出與模型預測變化幅度關聯最大的五個特徵。

Cell 26 — 前五個特徵係數表

Python 執行

```python
# ========================================
# 26. Top 5 Important Features
# ========================================

coef_df = pd.DataFrame({
    "Feature": selected_features,
    "Coefficient": ols_model.params.drop("const"),
    "P-value": ols_model.pvalues.drop("const")
})

coef_df["Abs Coefficient"] = (
    coef_df["Coefficient"].abs()
)

# 依標準化係數絕對值排序
coef_df = coef_df.sort_values(
    "Abs Coefficient",
    ascending=False
)

top5 = coef_df.head(5)

print("Top 5 Features:")
display(top5.round(6))

# 視覺化係數
plt.figure(figsize=(10, 5))

colors = [
    "steelblue" if c >= 0 else "salmon"
    for c in top5["Coefficient"]
]
```

```python
plt.barh(
    top5["Feature"],
    top5["Coefficient"],
    color=colors
)

plt.axvline(0, color="black", linewidth=0.8)
plt.gca().invert_yaxis()

plt.title("Top 5 Features by Absolute OLS Coefficient")
plt.xlabel("Standardized Feature Coefficient")
plt.ylabel("Feature")

plt.tight_layout()
plt.show()
```

Cell 27 — 自動產生前五個係數的中文解釋

Python 執行

```python
# ========================================
# 27. Interpret Top 5 Coefficients
# ========================================

print("=== Top 5 Feature Interpretation ===\n")

for rank, (feature, row) in enumerate(
    top5.iterrows(), start=1
):

    coef = row["Coefficient"]
    p_value = row["P-value"]

    direction = "增加" if coef > 0 else "減少"

    print(f"{rank}. {feature}")
    print(f"   Coefficient: {coef:.6f}")
    print(f"   P-value: {p_value:.6g}")

    print(
        f"   在其他變數固定下，"
        f"{feature} 增加 1 個原始標準差時，"
        f"模型預測的 FloodProbability "
        f"{direction} {abs(coef):.6f}。"
    )

    if p_value < 0.05:
        print("   Nominal p-value < 0.05")
    else:
        print("   Nominal p-value >= 0.05")
```

```
    print()
```

由於只有自變數 X 被標準化、目標 y 沒有標準化，係數的單位仍然是 `FloodProbability` 的數值單位。

例如，假設某特徵係數為 `0.008`，代表其他特徵不變、該特徵增加一個原始標準差時，模型預測的 FloodProbability 增加 0.008；若 FloodProbability 以 0–1 表示機率，則相當於增加 **0.8 個百分點**。

這是模型中的條件關聯，並不能直接推論「增加此特徵會導致洪水機率上升」。

---

5. 自動產生模型比較結論

Cell 28 — 輸出最佳模型及結論

Python 執行

```python
# ======================================
# 28. Modeling Summary
# ======================================

best_model_row = results_df_model.loc[
    results_df_model["R2"].idxmax()
]

print("=== Final Modeling Results ===")

print("Selected Features:", len(selected_features))

print("\nBest Predictive Model:")
print(best_model_row["Model"])

print("Test R2:", round(best_model_row["R2"], 6))
print("Test RMSE:", round(best_model_row["RMSE"], 6))
print("Test MAE:", round(best_model_row["MAE"], 6))

print("\nOLS Statistical Summary:")
print("Training R2:", round(ols_model.rsquared, 6))
print("Adjusted R2:", round(ols_model.rsquared_adj, 6))
print("Test R2:", round(r2_score(y_test, y_pred_ols), 6))

print("\nTop 5 Features:")
for feature in top5["Feature"]:
    print("-", feature)
```

6. CRISP-DM — Modeling 報告摘要

Modeling — Multiple Linear Regression

本研究使用前一階段特徵選擇所取得的最佳特徵組合，分別建立 LinearRegression、Ridge Regression 與 Lasso Regression 模型。

研究使用 80% 資料進行模型訓練，20% 資料進行最終測試，並使用 MAE、MSE、RMSE 及 R^2 比較模型預測表現。其中 Ridge 與 Lasso 透過交叉驗證選擇正則化參數，以評估正則化對模型效能的影響。

此外，本研究使用 statsmodels OLS 建立普通最小平方法迴歸模型，取得各特徵之迴歸係數、p-value、R^2 及 Adjusted R^2，以分析模型中各特徵與洪水機率之間的條件線性關係。

最後，依據標準化自變數的 OLS 係數絕對值，選出前五個重要特徵並解釋其係數方向與大小。

實驗結果：

- 最佳預測模型：［填入實際模型］
- 測試集 R^2：［填入數值］
- 測試集 RMSE：［填入數值］
- OLS Adjusted R^2：［填入數值］
- 前五個重要特徵：［填入特徵名稱］

本研究將根據測試集預測結果與迴歸係數分析，評估線性模型對洪水機率預測的適用性。由於資料集屬於合成資料，所得結果主要用於模型比較及方法學分析，不直接代表真實環境中的因果效應。

7. 方法學補充

有兩點值得在作業中說明：

第一，前五個係數不一定代表真正最重要的五個洪水因素。 係數可能受到特徵間相關性影響。如果希望更嚴謹地比較模型預測依賴哪些特徵，可以增加 Permutation Importance 分析。

第二，Ridge 與 Lasso 的交叉驗證應避免前處理資料洩漏。 上面 Cell 23 的 Pipeline 確保測試集沒有參與 `fit`，但 `RidgeCV` 與 `LassoCV` 的內部交叉驗證是在已經對整個訓練集標準化後進行的。若要求內層交叉驗證也完全無洩漏，應以 `GridSearchCV` 包覆包含 `SimpleImputer`、`StandardScaler` 與 `Ridge` / `Lasso` 的完整 Pipeline，讓每一折獨立擬合前處理步驟。

結論： 完成以上程式後，就能取
特徵的係數解釋。因為目前沒有

請幫我做模型評估：
1. 在測試集計算 MAE、MSE、RMSE、R^2、Adjusted R^2，整理成比較表
2. 畫「實際值 vs 預測值」散佈圖，加上 y=x 參考線
3. 用 statsmodels 的 get_prediction().summary_frame(alpha=0.05) 取得
   95% 信賴區間 (mean_ci) 與 95% 預測區間 (obs_ci)，
   抽 100 筆測試資料依預測值排序，畫出預測線 + 信賴區間 + 預測

區間陰影 + 實際值點
4. 殘差分析：殘差 vs 預測值圖、殘差直方圖、Q-Q plot、計算 VIF 檢查共線性
並說明是否符合線性回歸的假設
資料有一百多萬筆，畫圖時請抽樣避免太慢。

CRISP-DM — Evaluation（模型評估）

接續前面的 Colab Cell 28，本階段評估 LinearRegression、Ridge、Lasso 與 statsmodels OLS，並檢查線性回歸的模型假設。

評估設計

項目	評估方式
模型比較	MAE、MSE、RMSE、R^2、Adjusted R^2
預測能力	Actual vs Predicted 散佈圖
區間估計	OLS 95% Confidence Interval 與 Prediction Interval
殘差分析	Residual Plot、Histogram、Q-Q Plot
共線性	Variance Inflation Factor（VIF）

資料量處理原則： 評估指標使用完整測試集計算，不犧牲評估準確度。散佈圖、殘差圖與 Q-Q Plot 則使用固定亂數種子抽樣，避免百萬筆資料繪製造成 Colab 執行緩慢。

Cell 29 — 匯入套件與設定抽樣數量

Python 執行

```python
# ========================================
# CRISP-DM: Evaluation
# 29. Libraries and Settings
# ========================================

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm

from IPython.display import display
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
from statsmodels.stats.outliers_influence import (
    variance_inflation_factor
)
```

```python
# 圖表設定
sns.set_theme(style="whitegrid")
plt.rcParams["figure.dpi"] = 120

# 抽樣數量
PLOT_SAMPLE = 5000
INTERVAL_SAMPLE = 100
RANDOM_STATE = 42

# 確認前一階段模型已存在
assert len(selected_features) > 0
assert all(
    name in trained_models
    for name in ["LinearRegression", "Ridge", "Lasso"]
```

Cell 30 — 模型評估比較表

Adjusted R² 的計算方式為：

$$R^2_{\text{adj}} = 1 - (1 - R^2)\frac{n-1}{n-p-1}$$

其中 n 為樣本數，p 為模型中自變數的數量（不包含截距）。

Python 執行

```python
# 測試集實際值
y_true = np.asarray(y_test, dtype=float)

# 測試集樣本數
n = len(y_true)

# 使用相同數量的輸入特徵
p = len(selected_features)

assert n > p + 1

# 儲存各模型的預測
eval_predictions = {}

for name, model in trained_models.items():
    eval_predictions[name] = np.asarray(
        model.predict(X_test_selected),
        dtype=float
```

```
    )

# OLS 使用前面 Cell 25 建立的模型與設計矩陣
eval_predictions["OLS"] = np.asarray(
    ols_model.predict(X_test_ols),
    dtype=float
)

evaluation_results = []

for name, y_pred in eval_predictions.items():

    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)

    # 以測試集 R2 計算 Adjusted R2
    adj_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)

    evaluation_results.append({
        "Model": name,
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R2": r2,
        "Adjusted R2": adj_r2
    })

evaluation_df = pd.DataFrame(evaluation_results)

evaluation_df = evaluation_df.sort_values(
    "R2", ascending=False
).reset_index(drop=True)

print("=== Test Set Evaluation ===")
display(evaluation_df.round(6))

print("\nBest Model:")
print(evaluation_df.iloc[0]["Model"])
```

這裡的 Adjusted R^2 是將測試集 R^2 代入調整公式所得，方便作業比較，但它**不是傳統 OLS Summary 中基於訓練集的 Adjusted R^2**。對 Ridge 和 Lasso 而言，這也不是考慮正則化有效自由度後的嚴格調整值。

另外，由於 `LinearRegression` 與 `OLS` 使用相同資料與截距設定，兩者結果應幾乎一致。

Cell 31 — Actual vs Predicted 散佈圖

繪製四個模型的比較圖，只抽樣最多 5,000 筆測試資料。

Python 執行

```python
# ========================================
# 31. Actual vs Predicted
# ========================================

rng = np.random.default_rng(RANDOM_STATE)

# 隨機抽樣，不重複
sample_size = min(PLOT_SAMPLE, n)

sample_idx = rng.choice(
    n,
    size=sample_size,
    replace=False
)

fig, axes = plt.subplots(
    2, 2, figsize=(12, 10)
)

for ax, (name, y_pred) in zip(
    axes.flat,
    eval_predictions.items()
):

    actual_sample = y_true[sample_idx]
    predicted_sample = y_pred[sample_idx]

    ax.scatter(
        actual_sample,
        predicted_sample,
        alpha=0.25,
        s=12
    )

    # y=x 參考線
```

判讀方式： 散佈點越接近紅色 `y=x` 參考線，代表預測值越接近實際值。若出現系統性偏離，例如高實際值被低估、低實際值被高估，可能反映模型校準不足或極端值的回歸至平均數現象。

Cell 32 — 95% 信賴區間與預測區間

使用 statsmodels：

```python
ols_model.get_prediction(...).summary_frame(alpha=0.05)
```

取得下列統計量：

欄位	意義
`mean`	OLS 預測的平均值
`mean_ci_lower`	95% 平均預測信賴區間下限
`mean_ci_upper`	95% 平均預測信賴區間上限
`obs_ci_lower`	95% 個別觀測值預測區間下限
`obs_ci_upper`	95% 個別觀測值預測區間上限

兩者差異： 信賴區間描述某組特徵條件下「平均反應」估計的不確定性；預測區間則額外納入個別觀測值的隨機誤差，因此通常較寬。

Cell 32 — 抽樣 100 筆並繪製區間圖

```python
# ========================================
# 32. OLS Confidence and Prediction Intervals
# ========================================

# 隨機抽取最多 100 筆測試資料
rng_interval = np.random.default_rng(RANDOM_STATE)

interval_idx = rng_interval.choice(
    len(y_test),
    size=min(INTERVAL_SAMPLE, len(y_test)),
    replace=False
)

# 以位置索引取得 OLS 設計矩陣
X_interval = X_test_ols.iloc[interval_idx].copy()

# 取得實際目標值
y_interval = np.asarray(
    y_test.iloc[interval_idx],
```

```
    dtype=float
)

# 透過 statsmodels 計算預測區間
prediction = ols_model.get_prediction(X_interval)

prediction_frame = prediction.summary_frame(
    alpha=0.05
)

# 加入實際值
interval_df = prediction_frame.reset_index(drop=True).copy()
interval_df["Actual"] = y_interval

# 依預測平均值排序
interval_df = interval_df.sort_values(
```

Cell 33 — 計算預測區間涵蓋率

Python 執行

```python
# ======================================
# 33. Prediction Interval Coverage
# ======================================

# 判斷實際值是否落在 95% 預測區間
covered = (
    (interval_df["Actual"] >= interval_df["obs_ci_lower"]) &
    (interval_df["Actual"] <= interval_df["obs_ci_upper"])
)

coverage = covered.mean()

print("Number of Samples:", len(interval_df))
print(f"Empirical Prediction Interval Coverage: {coverage:.2%}")
print("Nominal Coverage: 95.00%")
```

這是 100 筆抽樣資料的經驗涵蓋率，不應期待剛好等於 95%，也不足以單獨確認區間校準程度。

此外，標準 OLS 預測區間依賴模型誤差假設。`FloodProbability` 是有界目標，OLS 預測與區間本身卻沒有強制限制在 0–1 之間，因此可能出現超出機率範圍的上下界。這不代表程式計算錯誤，而是 OLS 模型的一項限制。

---

4. 殘差分析（Residual Diagnostics）

殘差定義為：

$$e_i = y_i - \hat{y}_i$$

接下來使用測試集檢查模型預測誤差的分布。先以 OLS 為主要診斷對象，因為它有完整的傳統迴歸統計推論框架。

Cell 34 — 殘差 vs 預測值圖

Python 執行

```python
# ======================================
# 34. Residuals vs Fitted Values
# ======================================

# 完整測試集的 OLS 預測與殘差
ols_pred = eval_predictions["OLS"]
residuals = y_true - ols_pred

# 僅抽樣繪圖
pred_sample = ols_pred[sample_idx]
residual_sample = residuals[sample_idx]

plt.figure(figsize=(10, 6))

plt.scatter(
    pred_sample,
    residual_sample,
    alpha=0.25,
    s=12
)

plt.axhline(
    y=0,
    color="red",
    linestyle="--",
    linewidth=1.5
)

plt.title("Residuals vs Fitted Values")
plt.xlabel("Predicted Flood Probability")
plt.ylabel("Residuals")

plt.tight_layout()
plt.show()
```

判讀方式： 理想情況下，殘差應在 0 附近隨機分散，不應出現明顯曲線或漏斗形狀。

- 曲線型態：可能表示線性函數形式不足。
- 漏斗形狀：可能表示異質變異（Heteroscedasticity）。
- 殘差集中偏正或偏負：可能存在系統性預測偏差。

Cell 35 — 殘差直方圖

Python 執行

```python
# ======================================
# 35. Residual Distribution
# ======================================

plt.figure(figsize=(10, 6))

sns.histplot(
    residual_sample,
    bins=50,
    kde=True,
    color="steelblue"
)

plt.axvline(
    0,
    color="red",
    linestyle="--"
)

plt.title("Distribution of OLS Residuals")
plt.xlabel("Residuals")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

判讀方式： 觀察殘差是否以 0 為中心、是否對稱、是否存在偏態或厚尾。殘差接近常態有助於傳統 OLS 小樣本推論，但並不是 OLS 係數無偏的必要條件。

Cell 36 — Q-Q Plot

Python 執行

```python
# ======================================
# 36. Normal Q-Q Plot
# ======================================

plt.figure(figsize=(8, 7))

sm.qqplot(
    residual_sample,
    line="45",
    fit=True,
    ax=plt.gca(),
    markerfacecolor="steelblue",
    markeredgecolor="steelblue",
    alpha=0.4
)

plt.title("Normal Q-Q Plot of OLS Residuals")
```

```
plt.tight_layout()
plt.show()
```

判讀方式： 若點大致沿參考線排列，代表殘差分布與常態分布較接近。若尾端明顯偏離，可能存在厚尾、偏態或離群值。

由於本研究的目標變數有界，且樣本數極大，Q-Q Plot 即使出現偏離，也應結合實際偏離幅度判斷，不能單靠一張圖宣稱模型無效。

5. VIF — 多重共線性檢查

Variance Inflation Factor（變異膨脹因子）用來衡量某個特徵能被其他特徵線性解釋的程度。

$$VIF_j = \frac{1}{1 - R_j^2}$$

其中 R_j^2 是以第 j 個特徵作為應變數、其餘特徵作為自變數所得到的 R^2。

VIF	常見解讀
接近 1	幾乎沒有線性共線性
1–5	通常可接受
5–10	值得進一步檢查
超過 10	可能存在嚴重共線性

這些是經驗準則，不是絕對統計判定門檻。

Cell 37 — 計算 VIF

為了避免大量資料計算過慢，以下使用最多 20,000 筆**訓練資料**估計 VIF，不使用測試集。

Python 執行

```
# ========================================
# 37. Variance Inflation Factor
# ========================================

# 使用前面已標準化的 OLS 訓練資料
# 排除截距 const
X_vif = X_train_ols.drop(
    columns=["const"],
    errors="ignore"
)

# 最多抽取 20,000 筆訓練資料
```

```python
vif_sample_size = min(20000, len(X_vif))

X_vif_sample = X_vif.sample(
    n=vif_sample_size,
    random_state=42
)

# 計算各特徵的 VIF
vif_results = []

for i, feature in enumerate(X_vif_sample.columns):

    vif = variance_inflation_factor(
        X_vif_sample.to_numpy(),
        i
    )

    vif_results.append({
        "Feature": feature,
        "VIF": vif
    })

vif_df = pd.DataFrame(vif_results)
```

此程式使用的是抽樣後的標準化特徵矩陣，而且沒有加入截距；嚴格來說，這種計算在抽樣資料的欄位平均值不完全為 0 時，可能與傳統含截距的 VIF 略有不同。若要更嚴謹，建議改成以下版本，先加入截距後計算。

Cell 38 — 含截距的正式 VIF 計算

Python 執行

```python
# ========================================
# 38. VIF with Intercept
# ========================================

X_vif_design = sm.add_constant(
    X_vif_sample,
    has_constant="add"
)

vif_results = []

for i, feature in enumerate(X_vif_design.columns):
```

```
    if feature == "const":
        continue

    vif = variance_inflation_factor(
        X_vif_design.to_numpy(),
        i
    )

    vif_results.append({
        "Feature": feature,
        "VIF": vif
    })

vif_df = pd.DataFrame(vif_results)

vif_df = vif_df.sort_values(
    "VIF", ascending=False
).reset_index(drop=True)

display(vif_df.round(4))

# VIF 視覺化
```

以 Cell 38 的結果作為正式 VIF 報告即可。 Cell 37 可視為初步檢查。

6. 線性回歸假設檢查與結果解釋

Cell 39 — 統整診斷數據

Python 執行

```
# ======================================
# 39. Regression Diagnostics Summary
# ======================================

from scipy.stats import skew, kurtosis

print("=== Regression Diagnostics ===")

print("\nModel Performance:")
ols_row = evaluation_df[
    evaluation_df["Model"] == "OLS"
].iloc[0]

print("Test R2:", round(ols_row["R2"], 6))
print("Test RMSE:", round(ols_row["RMSE"], 6))

print("\nResidual Statistics:")
print("Mean:", round(residuals.mean(), 6))
print("Standard Deviation:", round(residuals.std(), 6))
print("Skewness:", round(skew(residuals), 6))
print("Excess Kurtosis:", round(kurtosis(residuals), 6))

print("\nMulticollinearity:")
print("Maximum VIF:", round(vif_df["VIF"].max(), 4))
print("Features with VIF > 5:", int((vif_df["VIF"] > 5).sum()))
print("Features with VIF > 10:", int((vif_df["VIF"] > 10).sum()))

print("\nOLS Training Statistics:")
print("R2:", round(ols_model.rsquared, 6))
print("Adjusted R2:", round(ols_model.rsquared_adj, 6))
```

如何判斷是否符合 OLS 假設？

假設	檢查方式	判斷原則
線性關係	Residuals vs Fitted	無明顯曲線或系統性型態
誤差變異數恆定	Residuals vs Fitted	不呈現明顯漏斗形
殘差常態性	Histogram、Q-Q Plot	分布接近常態
無嚴重共線性	VIF	數值越低通常越好
誤差獨立性	資料來源與抽樣設計	不能僅憑上述圖表確認
外生性	研究設計、遺漏變數與資料產生機制	無法單靠殘差圖證明

目前不能直接宣稱 OLS 假設全部成立。 必須檢視執行後的圖表與統計數據，而且誤差獨立性、外生性等假設還需要資料產生過程的資訊才能評估。

尤其本資料集是合成資料，應將統計結果視為資料集內的模型分析，而不是現實世界洪水機率機制的驗證。

7. CRISP-DM — Evaluation 報告摘要

Evaluation — Model Performance and Diagnostics

本研究使用獨立測試集評估 LinearRegression、Ridge Regression、Lasso Regression 與 statsmodels OLS 模型的預測效能，並以 MAE、MSE、RMSE、R^2 及 Adjusted R^2 進行比較。

為避免大量資料影響繪圖效率，所有模型評估指標均使用完整測試集計算，而視覺化分析採用隨機抽樣方式進行。

研究首先繪製實際值與預測值散佈圖，觀察模型預測與理想預測線的接近程度。接著使用 statsmodels 的 `get_prediction().summary_frame(alpha=0.05)`，取得 OLS 平均預測值的 95% 信賴區間及個別觀測值的 95% 預測區間，並視覺化呈現模型預測的不確定性。

最後，透過殘差與預測值散佈圖、殘差直方圖、Q-Q Plot 及 VIF，評估線性模型的函數形式、誤差分布及多重共線性情形。

實驗結果：

- 最佳模型：［填入模型名稱］
- 測試集 R^2：［填入數值］
- 測試集 RMSE：［填入數值］
- OLS Adjusted R^2（訓練集）：［填入數值］
- 最大 VIF：［填入數值］
- 殘差常態性：［根據 Q-Q Plot 結果說明］
- 等變異性：［根據 Residuals vs Fitted 結果說明］

綜合模型評估指標與殘差診斷結果，討論線性迴歸對本資料集的預測效能及適用限制。

最後的模型選擇建議

在沒有實際執行結果前，**無法可靠地判定 LinearRegression、Ridge 或 Lasso 哪個最好**。

建議以完整測試集的 R^2、RMSE 為主要比較依據，輔以 MAE、殘差型態及模型複雜度。若三者表現非常接近，可以優先考慮較容易解釋的 LinearRegression；若正則化模型有穩定且實質的預測改善，則可考慮 Ridge 或 Lasso。

另須注意，測試集應作為最終評估，而非反覆依照測試結果調整模型。若希望在定案前比較模型，應優先使用訓練集內的交叉驗證結果。

至此，作業已涵蓋 CRISP-DM 的
Preparation、Modeling 與 Eva
成整套流程。

請幫我：
1. 用全部訓練資料重新訓練最終模型，對 test.csv 預測，輸出 submission.csv
2. 用 joblib 儲存模型

3. 寫一個 predict_flood(輸入資料) 函式示範如何預測新資料
4. 用一段文字說明這個模型實際上可以怎麼部署使用

CRISP-DM — Deployment（模型部署與應用）

這是整份 Kaggle Playground Series S4E5 洪水預測作業的最後階段，接續前面的 Colab Cell 39。

本階段將完成：

1. 使用 `train.csv` 中全部可用的訓練資料，重新訓練最終模型。
2. 對 `test.csv` 產生預測，輸出符合 Kaggle 格式的 `submission.csv`。
3. 使用 `joblib` 儲存包含資料前處理的完整模型。
4. 建立 `predict_flood()` 函式，示範如何輸入新資料進行預測。
5. 說明模型如何部署成實際應用系統。

重要： 這次的「全部訓練資料」指 `train.csv` 中的所有可用資料，而不只是先前切分出的 80% `X_train`。原先保留的 20% 已完成評估，現在才將其併回正式訓練資料。

Cell 40 — 確認最終模型與特徵

沿用之前 Feature Selection 的 `best_features`，以及 Modeling 階段訓練的模型。

Python 執行

```python
# ========================================
# CRISP-DM: Deployment
# 40. Final Model Configuration
# ========================================

import pandas as pd
import numpy as np
import joblib

from sklearn.base import clone
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso

# 前面特徵選擇階段得到的特徵
final_features = list(best_features)

# 以之前測試集評估中表現最好的模型為例
best_model_name = evaluation_df.iloc[0]["Model"]

# statsmodels OLS 與 sklearn LinearRegression
# 本質上都是普通最小平方法
if best_model_name == "OLS":
    best_model_name = "LinearRegression"

print("Final Model:", best_model_name)
print("Number of Features:", len(final_features))
```

```
print("Selected Features:", final_features)
```

此處沿用前一步已決定的最佳模型。嚴格的機器學習實驗應在查看測試集結果之前，透過訓練集交叉驗證決定最終模型種類；若前面的最佳模型是根據測試集挑選，報告中應註明測試集已參與模型選擇。

Cell 41 — 使用全部 train.csv 重新訓練模型

這裡使用 `Pipeline` 將缺失值填補、標準化及迴歸模型整合，確保之後只需要呼叫一次 `predict()`。

對 Ridge 和 Lasso，沿用前一階段選出的最佳 alpha，而不是重新使用測試集調整參數。

Python 執行

```python
# ========================================
# 41. Retrain Final Model
# ========================================

# 使用 train.csv 的全部可用資料
df_final = df.drop(
    columns=["id"],
    errors="ignore"
).copy()

# 去除完全重複資料
df_final = df_final.drop_duplicates()

# 去除目標值缺失的資料
df_final = df_final.dropna(
    subset=["FloodProbability"]
)

# 選取最終特徵與目標
X_full = df_final[final_features].copy()
y_full = df_final["FloodProbability"].copy()

# 根據先前選出的模型類型建立新模型
if best_model_name == "LinearRegression":
    estimator = LinearRegression()

elif best_model_name == "Ridge":
    best_alpha = trained_models["Ridge"].named_steps["model"].alpha_
    estimator = Ridge(alpha=best_alpha)

elif best_model_name == "Lasso":
    best_alpha = trained_models["Lasso"].named_steps["model"].alpha_
    estimator = Lasso(
        alpha=best_alpha,
        max_iter=10000,
```

這裡的重新訓練不會改變前面取得的最終測試集分數，但它會產生新的模型係數，因此後續的正式預測應使用 `final_model`，而非前面的 `trained_models`。

Cell 42 — 讀取 test.csv 並產生 submission.csv

已確認 Kaggle S4E5 官方要求的提交格式為 `id,FloodProbability`，且每個測試集 ID 都必須有一個預測值。 Kaggle

Python 執行

```python
# ========================================
# 42. Kaggle Submission
# ========================================

from google.colab import files
import io

# 上傳 test.csv
uploaded_test = files.upload()

if "test.csv" not in uploaded_test:
    raise FileNotFoundError("Please upload test.csv")

test_df = pd.read_csv(
    io.BytesIO(uploaded_test["test.csv"])
)

# 檢查必要欄位
assert "id" in test_df.columns

missing_columns = [
    col for col in final_features
    if col not in test_df.columns
]

if missing_columns:
    raise ValueError(
        f"Missing feature columns: {missing_columns}"
    )

# 保留訓練時的特徵順序
```

```python
X_kaggle = test_df[final_features].copy()

# 使用最終模型預測
test_predictions = final_model.predict(X_kaggle)
```

此處保留線性迴歸模型的原始預測值，不自動截斷至 0–1，避免無提示地改變已評估模型的預測結果。若預測值超出此範圍，應另外檢查其比例，並評估是否需要有界輸出模型。

Cell 43 — 使用 joblib 儲存模型

不只儲存迴歸係數，而是將整個 Pipeline 儲存，確保之後載入模型仍可自動完成缺失值填補、標準化及預測。

Python 執行

```python
# ========================================
# 43. Save Model with joblib
# ========================================

import joblib

model_package = {
    "model": final_model,
    "features": final_features,
    "model_name": best_model_name,
    "target": "FloodProbability"
}

# 儲存模型及相關資訊
joblib.dump(
    model_package,
    "flood_prediction_model.joblib"
)

print("Model saved successfully!")
```

```python
# 測試重新載入
loaded_package = joblib.load(
    "flood_prediction_model.joblib"
)

loaded_model = loaded_package["model"]

# 比較儲存前後的預測結果
original_pred = final_model.predict(
    X_full.head(5)
)

loaded_pred = loaded_model.predict(
    X_full.head(5)
```

`joblib` 檔案可能包含 Python 物件序列化內容，因此只應載入自己建立或信任來源的模型檔案，並盡量保持 scikit-learn 等套件版本一致。

Cell 44 — 建立 predict_flood() 函式

這個函式接受單筆 Python `dict`、多筆 `list[dict]` 或 pandas DataFrame。

Python 執行

```python
# ========================================
# 44. Prediction Function
# ========================================

def predict_flood(input_data):
    """
    使用已儲存模型預測洪水機率。

    Parameters
    ----------
    input_data : dict, list[dict], or DataFrame
        包含模型所需特徵的資料。

    Returns
    -------
    pandas.DataFrame
        FloodProbability 預測結果。
    """

    package = joblib.load(
        "flood_prediction_model.joblib"
```

```
    )

    model = package["model"]
    features = package["features"]

    # 單筆資料
    if isinstance(input_data, dict):
        data = pd.DataFrame([input_data])

    # 多筆資料
    elif isinstance(input_data, list):
        data = pd.DataFrame(input_data)

    # DataFrame
```

Cell 45 — 示範預測新資料

為了避免虛構特徵名稱或超出資料集合理範圍的輸入值，以下使用 `train.csv` 的第一筆特徵資料模擬一筆新輸入。

這只是在驗證函式能正常運作，不代表該筆資料可用於獨立測試。

Python 執行

```
# ========================================
# 45. Example Prediction
# ========================================

# 示範輸入，使用資料集中的一筆特徵
example_input = X_full.iloc[0].to_dict()
```

```
print("Example Input:")
print(example_input)

# 執行預測
result = predict_flood(example_input)

print("\nPredicted Flood Probability:")
display(result)

# 多筆預測範例
batch_input = X_full.head(5)

batch_result = predict_flood(batch_input)

print("\nBatch Predictions:")
display(batch_result)
```

6. 模型部署應用說明

Deployment — Flood Probability Prediction System

本研究完成多元線性回歸模型的訓練與評估後，使用全部訓練資料重新建立最終模型，並將資料前處理及預測程序整合為 Pipeline，再透過 joblib 儲存，以便後續重複使用。

實際部署時，可使用 Python Flask 或 FastAPI 建立預測 API，或透過 Streamlit 開發網頁操作介面。使用者輸入環境及人為風險因素後，系統便可自動完成資料前處理，並輸出 FloodProbability 預測結果。此外，也可結合資料庫進行批次分析、儲存預測紀錄及製作視覺化儀表板。

在實務應用上，洪水風險預測可作為防災規劃、基礎設施改善及資源分配的輔助分析工具。然而，本研究使用 Kaggle 合成資料集，模型尚未經過真實地區、歷史洪水事件與實際監測資料驗證，因此目前適合用於機器學習教學及系統原型展示，不宜直接作為真實洪水預警或公共安全決策依據。

7. 最終產出

執行上述 Colab 程式後，會產生兩個檔案：

submission.csv

Kaggle 提交檔案，包含 `id` 與 `FloodProbability` 兩個欄位。

flood_prediction_model.joblib

包含完整前處理 Pipeline、訓練後模型、特徵名稱及模型資訊。

這兩個檔案需在你的 Colab 執行後產生；目前未取得你的 `train.csv`、`test.csv` 或實際模型，因此尚未生成可直接下載的成品。

競賽狀態提醒： Kaggle Playground Series S4E5 的正式競賽已於 2024 年 5 月 31 日截止。現在仍可使用該資料集完成課程作業，但是否能進行 Late Submission，需以 Kaggle 頁面實際提供的功能為準。 k Kaggle

至此，整份作業已完成 CRISP-DM 六個階段的程式架構：Business Understanding、Data Understanding、Data Prepara

這個競賽網路上主流的高分做法有哪些？（例如把特徵加總、統計特徵、
LightGBM/XGBoost、集成模型）請和我的線性回歸結果比較，
用表格整理優缺點，並說明為什麼線性回歸在這題表現已經不錯。

處理時間為 39s

Kaggle S4E5 Flood Prediction：高分解法分析

這場競賽最重要的發現是：特徵工程往往比換用更複雜的機器學習模型重要。 尤其是將 20 個特徵加總，建立 `feature_sum`，這是公開高分解法反覆使用的核心技巧。

根據 Kaggle 官方競賽討論、前幾名選手的解法與公開 Notebook，這題的高分做法主要包括：

- **特徵加總（Sum）**：建立 20 個特徵的總和。
- **統計特徵（Statistical Features）**：平均數、標準差、最大值、最小值、偏態等。
- **LightGBM / XGBoost / CatBoost**：搭配上述特徵工程。
- **Ensemble / Stacking**：結合多個模型的預測結果，進一步提高 R^2。

競賽官方採用 R^2 評分，於 2024 年 5 月 31 日結束。 k Kaggle +1

1. 公開高分解法的實際成績

公開第一名 Notebook
0.86943
Ridge Ensemble，公開顯示分數

LightGBM Notebook
0.86931
特徵工程 + LightGBM

以上為 Kaggle 公開 Notebook 頁面所顯示的分數，不應直接視為你的交叉驗證 R^2，也不能混用 Public 與 Private Leaderboard 比較。 k Kaggle

公開方法	R^2	結果來源
第一名解法 Notebook：Ridge Ensemble	0.86943	Kaggle 高分 Notebook ↗
LightGBM + Feature Engineering	0.86931	同上，公開 Notebook
XGBoost + 排序特徵 + Sum	0.86908 ± 0.00055	Kaggle 討論（CV 結果） ↗
LightGBM + 統計特徵	0.86919	公開專案（5-fold CV） ↗
Linear Regression，該公開專案基準	約 0.845	同一 GitHub 專案

最後一項來自該專案對資料結構與基準模型的分析，不代表所有線性回歸模型都一定只能得到 0.845。不同資料切分、特徵處理與評估方式可能得到不同數值。 k Kaggle +1

2. 為什麼特徵加總如此重要？

公開競賽討論指出，這個合成資料集的 20 個特徵可能具有近似對等的統計角色，目標值主要由這些特徵的加總及其他變化所決定。這是選手根據資料分析提出的解釋，不應視為官方已完整公開的資料生成公式。 Kaggle

可以用一個簡化模型理解：

$$\hat{y} \approx \beta_0 + c\sum_{j=1}^{20} x_j$$

它與你的多元線性回歸：

$$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_{20} x_{20}$$

有非常重要的關係：

當所有迴歸係數接近相同時，20 個特徵的多元線性回歸，就會近似於一個使用特徵總和的簡單線性回歸。

這也解釋了為什麼：

- 線性回歸可以直接學到加總型的線性關係。
- LightGBM、XGBoost 等樹模型，如果沒有明確加入 Sum，反而需要透過大量樹分裂去逼近加法關係。
- 加入 Sum 之後，樹模型更容易捕捉主要訊號，並利用其他統計特徵建模殘餘的非線性變化。

因此，複雜模型在這題的優勢並不是單純來自演算法複雜，而是來自適當的特徵表示。

3. 各種高分方法的優缺點

方法	優點	缺點	本題評價
LinearRegression（你的方法）	訓練快、容易解釋、能直接捕捉線性加總關係	無法自行捕捉非線性或交互作用	很適合當基準模型
LinearRegression + Sum	模型簡單，直接反映總風險分數	Sum 是原始特徵的線性組合，與全部原始特徵一起使用會產生完全共線性	值得做對照實驗
Sum + 統計特徵	容易實作，可提供更多分布資訊	特徵可能重複，未必全部有效	高優先度
LightGBM	訓練效率較高，可捕捉非線性關係	缺少 Sum 時可能不擅長學習整體加總結構	本題高分主力
XGBoost	非線性建模能力強，可調整正則化	訓練與調參成本較高	有公開高分實例
CatBoost	適合表格資料，可捕捉複雜關係	本題沒有明顯需要發揮的類別特徵優勢	可作集成成員

Ensemble / Stacking	能整合不同模型，減少部分預測誤差	計算成本高、易因不當驗證過度擬合	最後微調分數

Kaggle 第二名隊伍的公開解法使用 56 個模型，包含 XGBoost、CatBoost、LightGBM 等，並透過 OOF（Out-of-Fold）預測、LinearRegression 加權與 Forward Feature Selection 組成集成模型。他們表示最好的單一模型約為 0.86933，後續才透過集成提高成績。 k Kaggle

這說明在這個競賽中，**頂尖分數的最後改善幅度可能非常有限，但需要投入大量的計算和模型調整工作。**

4. 與你的線性回歸結果比較

目前對話中尚未提供你執行 Colab 後的實際 R^2、RMSE，因此無法直接宣稱你的模型已經達到 0.84 或 0.86。

以下是可以確認的比較：

比較項目	你的模型	公開高分方法
原始特徵	選取 5 / 10 / 15 / 20 個	通常包含原始特徵與工程特徵
特徵選擇	SelectKBest、RFE、LassoCV	聚合特徵、排序特徵、模型選擇
演算法	Linear、Ridge、Lasso	LightGBM、XGBoost、CatBoost、Ensemble
R^2	尚未提供執行結果	高分公開 Notebook 約 0.8693–0.8694
可解釋性	高	視模型而定
運算成本	低至中	中至高
適合的研究目的	解釋特徵與線性關係	追求競賽預測分數

值得注意的是，你前面使用特徵選擇時，如果最終只保留 5 或 10 個特徵，可能反而失去部分加總資訊。

在這份資料中，**即使單一特徵的迴歸係數或相關係數不突出，20 個特徵合計仍可能具有很強的預測能力**。因此我不建議單純為了減少特徵數，就直接認定 5 個或 10 個特徵優於全部 20 個特徵。

5. 為什麼線性回歸在這題可以表現很好？

可以歸納成三個核心原因。

第一，資料的主要訊號近似線性加總。 當目標主要依賴 20 個特徵的總和時，線性回歸的函數形式正好適合描述這類關係，無須透過大量非線性分裂去逼近。

第二，資料量足夠大。 這場競賽有約 111.8 萬筆訓練資料，但只有 20 個原始特徵。相較於模型參數數量，可用樣本非常多，有助於穩定估計線性迴歸係數。 GitHub

第三，非線性模型的新增價值相對有限。 競賽高分方法雖能進一步捕捉非線性和統計分布特徵，但從公開結果來看，其優勢往往建立在特徵工程之上，而不是僅僅替換演算法。 k Kaggle +1

6. 我建議你在 Colab 額外做的簡單實驗

不用重新建立複雜的 LightGBM，也能測試這場競賽最重要的發現。

直接比較：

- A：你先前選出的最佳線性回歸特徵
- B：全部 20 個原始特徵
- C：只使用 20 個特徵的加總 `feature_sum`

以下程式直接接續前面的 Colab，使用相同的 80/20 切分進行比較。

Python 執行

```python
# ======================================
# Additional Experiment:
# Linear Regression vs Feature Sum
# ======================================

from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score, mean_squared_error
import numpy as np
import pandas as pd

# 原始 20 個特徵
all_features = X_train.columns.tolist()

# 防止選到 ID 或目標欄位
all_features = [
    c for c in all_features
    if c not in ["id", "FloodProbability"]
]

assert len(all_features) == 20

# 先保留原始資料
Xtr_all = X_train[all_features].copy()
Xte_all = X_test[all_features].copy()

# 使用訓練集中位數填補缺失值
imputer = SimpleImputer(strategy="median")

Xtr_imp = pd.DataFrame(
    imputer.fit_transform(Xtr_all),
    columns=all_features,
    index=Xtr_all.index
```

這個實驗非常適合納入你的作業 Discussion，因為它可以直接檢驗「複雜的特徵選擇是否真的比保留全部特徵好」。

但若這些模型都是看過同一份測試集後才新增比較的，就應視為探索性分析，而非完全獨立的最終模型評估。

7. 可直接放入作業的 Discussion

Discussion — Comparison with Kaggle High-Scoring Solutions

本研究使用 Multiple Linear Regression、Ridge Regression 與 Lasso Regression 預測 FloodProbability，並與 Kaggle Playground Series S4E5 公開高分解法進行比較。

根據 Kaggle 競賽討論與公開 Notebook，高分模型普遍採用特徵工程，包括將 20 個原始特徵加總，以及建立平均數、標準差、最大值、最小值等統計特徵，再搭配 LightGBM、XGBoost、CatBoost 或 Ensemble Models 提升預測表現。部分公開高分 Notebook 的 R^2 約為 0.8693 至 0.8694。

本資料集的一項重要特性，是洪水機率與各特徵的總和存在很強的關聯。因此，多元線性回歸能夠有效捕捉主要的線性加總結構，不一定需要複雜的非線性模型才能建立具有競爭力的基準預測結果。

相較於梯度提升樹及集成模型，線性回歸具有訓練成本低、模型結構簡單、迴歸係數容易解釋等優點，適合作為本研究的基準模型。

然而，線性回歸無法直接捕捉非線性關係與特徵交互作用，因此在追求競賽最高分數時，加入統計特徵並搭配梯度提升樹及模型集成，仍可能進一步提升預測表現。

綜合而言，本研究顯示，模型複雜度並非預測效能的唯一決定因素。選擇符合資料結構的模型，並透過適當的特徵工程與交叉驗證進行評估，是建立有效預測模型的重要原則。

最值得放在作業結論的一句話：

> 本研究顯示，當資料的主要預測訊號具有近似線性加總結構時，多元線性回歸即能有效捕捉主要變異，而複雜模型的進一步改善則仰賴特徵工程及非線性關係的建模。

這個結論有公開競賽解法作為依據，也能直接連結你前面完成的 CRISP-DM 分析。



5115056015_hw2.ipynb
程式碼

這是我的程式碼與結果，然後附上kaggle成果
幫我照這個目錄寫成word報告，每節放對應的圖和說明：

1. **資料集來源**：名稱、Kaggle 網址、筆數、20 個特徵是什麼
2. **Business Understanding**：為什麼預測洪水機率有用
3. **Data Understanding**：描述統計、分布圖、相關熱力圖
4. **Data Preparation**：缺失值、切分、標準化
5. **Modeling**：特徵選擇比較圖表 + 選擇理由、模型係數解讀
6. **Evaluation**：指標比較表、實際 vs 預測圖、**預測區間圖**、殘差圖、Kaggle 分數截圖
7. **Deployment**：如何使用模型
8. **網路主流解法比較**：表格（你的線性回歸 vs 特徵加總技巧 vs 樹模型）

處理時間為 3m 7s

已依照你指定的 **8 個章節**，完成 Word 報告，內容以你提供的 Colab Notebook 實際執行結果與 Kaggle 成績截圖為依據。

Flood Prediction — CRISP-DM 完整實驗報告

Word（.docx）| 14 頁 | 包含實驗圖表、數據比較及分析

下載 Word 完整報告 ↗

報告內容

已依照目錄整理：

1. **資料集來源**：Kaggle 網址、1,117,957 筆資料及 20 個特徵說明。
2. **Business Understanding**：洪水預測的社會價值與研究目的。
3. **Data Understanding**：描述統計、目標分布、相關係數熱力圖。
4. **Data Preparation**：資料清理、80/20 切分、標準化。
5. **Modeling**：三種特徵選擇方法、交叉驗證結果及前五個迴歸係數。
6. **Evaluation**：模型評估表、Actual vs Predicted、95% 預測區間、殘差圖、Q-Q Plot、VIF，以及 Kaggle 成績截圖。
7. **Deployment**：模型儲存、預測函式與實際部署方式。
8. **網路主流解法比較**：線性回歸、特徵加總、LightGBM、XGBoost 及 Ensemble。

已納入的實際成績

指標	結果
5-Fold CV R^2	0.844959
最終模型	Ridge（20 個特徵）
測試集 R^2	0.844877
測試集 RMSE	0.020080
Kaggle Public Score	0.84458
Kaggle Private Score	0.84426

報告中的圖表均取自你上傳的 Notebook，沒有重新製造實驗數據。網路高分解法則另外列出參考來源，並與你的結果區分。

報告的核心結論： 保留全部 20 個特徵比僅保留 5、10 或 15 個特徵有明顯較好的預測表現，顯示本題的預測訊號分散於多個變數。線性模型已能捕捉主要的加總關係，而特徵工程與非線性模型是後續提升 Kaggle 分數的主要方向。